Quantum Kaizen

Deployment Guide

Version: 1.0

Updated: 2026-05-16

Audience: Developers deploying from scratch

Time required: ~30 minutes

Netlify Render Neon Upstash

Quantum Kaizen — Deployment Guide

Version: 1.0 · **Last updated:** 2026-05-16 **Audience:** Developers deploying Quantum Kaizen to production from scratch.

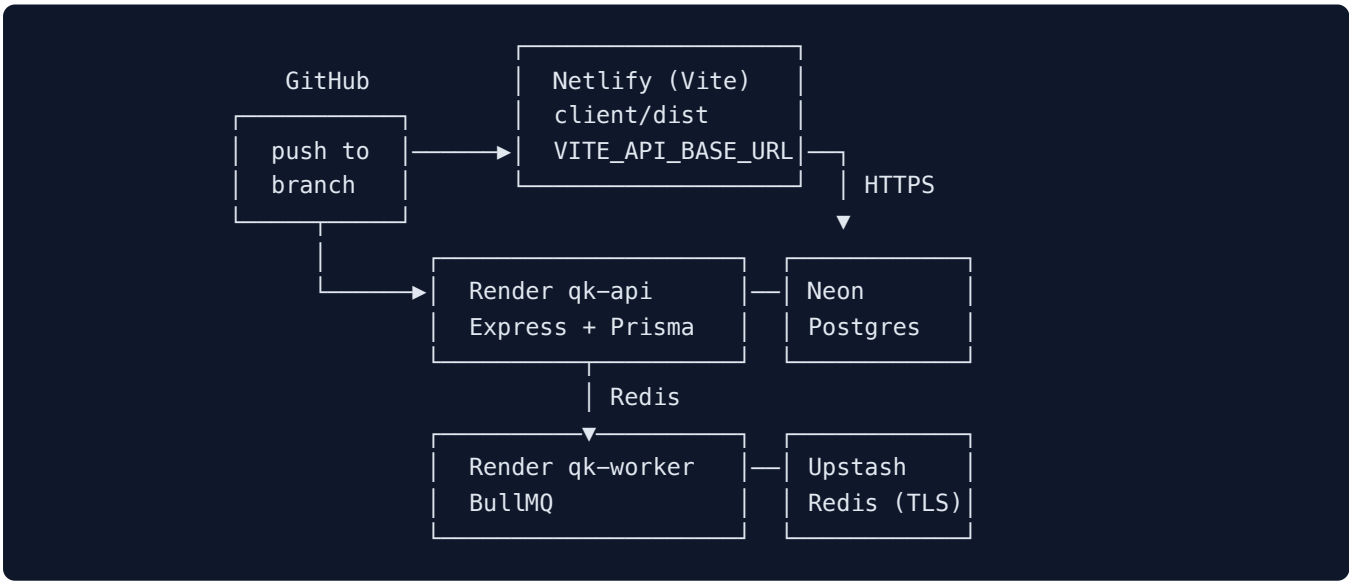
This guide walks through every account creation, every click, and every environment variable needed to put Quantum Kaizen on the internet. Allow ~30 minutes end to end.

---

1. Architecture

Layer	Provider	Why
Source control	**GitHub**	Render and Netlify both deploy on push
Frontend (SPA)	**Netlify**	Static hosting + CDN + custom domain (free tier)
Backend API	**Render**	Node web service, push-to-deploy
BullMQ worker	**Render**	Same Blueprint as the API; runs SLA + approval crons
PostgreSQL	**Neon**	Serverless Postgres, free tier
Redis (BullMQ)	**Upstash**	Serverless Redis with TLS, free tier

Cost on free tiers: $0/month. Render free services sleep after 15 min idle (~30 s cold start). Upgrade to Render Starter ($7/month per service) to eliminate this.



---

2. Prerequisites

Create free accounts on each provider before starting. All five accept GitHub OAuth — sign up with GitHub for the fastest path.

- GitHub: https://github.com/signup
- Neon: https://console.neon.tech (Sign up → Continue with GitHub)
- Upstash: https://console.upstash.com (Login → Continue with GitHub)
- Render: https://dashboard.render.com/register (Click GitHub)
- Netlify: https://app.netlify.com/signup (Click GitHub)

Tools needed on your laptop:

- `git`
- `openssl` (ships with macOS/Linux; on Windows use Git Bash)
- A code editor

---

3. Step 1 — Push the code to GitHub

Render and Netlify both deploy *from* GitHub. The code must be on GitHub first.

3.1 Create a GitHub repo

1. Go to https://github.com/new
2. **Repository name**: `quantumkaizen`
3. **Privacy**: Private (recommended)
4. Leave everything else blank — do **not** initialise with README/license
5. Click **Create repository**

3.2 Push from your laptop

From the project root:

```
git remote -v
# If no remote, add yours:
git remote add origin https://github.com/__BODY__lt;YOUR-ORG-OR-USERNAME__BODY__gt;/quantumka

git add .
git commit -m __BODY__quot;Initial deploy config__BODY__quot;
git push -u origin __BODY__lt;YOUR-BRANCH-NAME__BODY__gt;
```

Note the branch name you push — Render and Netlify will both deploy from it. In this guide we__BODY__#39;ll call it `__BODY__lt;DEPLOY_BRANCH__BODY__gt;`.

---

4. Step 2 — Provision PostgreSQL (Neon)

4.1 Create a Neon project

1. Open https://console.neon.tech
2. Click **New Project**
3. **Project name**: `quantum-kaizen`
4. **Postgres version**: `16`
5. **Region**: **AWS US East (N. Virginia)** (closest to Render__BODY__#39;s free tier)
6. Click **Create project**

4.2 Copy two connection strings

On the project Dashboard → **Connection Details** card, you__BODY__#39;ll see a URL and a __BODY__quot;Pooled connection__BODY__quot; toggle.

You need **both** versions of the URL.

A. Pooled URL (`DATABASE_URL`)

1. Tick the ☑ **Pooled connection** checkbox
2. Click **Copy**
3. Save to a temporary text editor. It looks like:

   ```
   postgresql://neondb_owner:npg_xxxx@ep-xxxxx-pooler.us-east-1.aws.neon.tech/neondb?sslmode=
   ```

   Note the `-pooler` in the hostname.

B. Direct URL (`DIRECT_URL`)

1. **Untick** the Pooled connection checkbox — the URL below changes
2. Click **Copy**
3. Save it. Same shape, but **no** `-pooler` in the hostname:

   ```
   postgresql://neondb_owner:npg_xxxx@ep-xxxxx.us-east-1.aws.neon.tech/neondb?sslmode=require
   ```

Critical: both must start with `postgresql://` (or `postgres://`) and end with `?sslmode=require` . Keep them safe — you__BODY__#39;ll paste them into Render in step 6.

4.3 First-time migration

If this is a fresh Neon project (no tables yet), apply all migrations once from your laptop:

```
cd backend
DATABASE_URL=__BODY__#39;__BODY__lt;your-pooled-url__BODY__gt;__BODY__#39; \
DIRECT_URL=__BODY__#39;__BODY__lt;your-direct-url__BODY__gt;__BODY__#39; \
  npx prisma migrate deploy
```

After this, every Render deploy runs `prisma migrate deploy` automatically.

---

5. Step 3 — Provision Redis (Upstash)

5.1 Create a Redis database

1. Open https://console.upstash.com
2. Click **Create Database**
3. **Name**: `qk-redis`
4. **Type**: **Regional** (cheaper than Global, fine for BullMQ)
5. **Region**: `us-east-1` (match Neon + Render)
6. **TLS**: ON (default)
7. Click **Create**

5.2 Copy the TCP connection string

BullMQ needs the **native Redis protocol** (TCP), not Upstash__BODY__#39;s HTTP REST API.

1. On the database page → scroll to **Connect to your database**
2. Click the **TCP** tab (next to __BODY__quot;REST__BODY__quot;)
3. Copy the full `rediss://` URL. It looks like:

```
rediss://default:gQAAAAAA...@xxxx.upstash.io:6379
```

Save it as `REDIS_URL`.

Important: the scheme must be `rediss://` (with two `s` — TLS). `redis://` will fail because Upstash requires TLS.

---

6. Step 4 — Deploy backend on Render (Blueprint)

Render__BODY__#39;s Blueprint feature reads `render.yaml` from the repo and provisions both services in one click.

6.1 Generate a JWT secret

On your laptop, run:

```
openssl rand -base64 48
```

Copy the output. Treat it like a password — don__BODY__#39;t paste it into chat/Slack/anywhere shareable.

6.2 Create the Blueprint

1. Open https://dashboard.render.com
2. Click **New +** (top right) → **Blueprint**
3. If prompted, click **Connect GitHub account** and authorise Render to read your repo
4. Pick **quantumkaizen** from the list
5. **Branch**: select `__BODY__lt;DEPLOY_BRANCH__BODY__gt;` (the branch you pushed in step 3.2)
6. **Blueprint Name**: `quantum-kaizen`
7. **Blueprint Path**: leave as `render.yaml`
8. Click **Apply**

Render parses `render.yaml` and proposes two services:

- `qk-api` — web service (Node, free plan)
- `qk-worker` — background worker (Node, free plan)

6.3 Paste the env vars

Render prompts for the secrets marked `sync: false` in `render.yaml`. You__BODY__#39;ll fill in **two cards** — one for `qk-api`, one for `qk-worker`. The values are identical except `qk-worker` doesn__BODY__#39;t ask for `CORS_ORIGIN`.

Variable	Value
`DATABASE_URL`	Neon **pooled** URL from step 4.2.A
`DIRECT_URL`	Neon **direct** URL from step 4.2.B
`REDIS_URL`	Upstash `rediss://...` from step 5.2
`JWT_SECRET`	Output of `openssl rand -base64 48` from step 6.1
`CORS_ORIGIN`	`*` for now — locked down in step 8

Paste rules (very common mistakes):

- ❌ No surrounding quotes: paste `postgresql://...` not `__BODY__quot;postgresql://...__BODY__quot;`
- ❌ No `KEY=` prefix: paste the value only, not `DATABASE_URL=postgres://...`
- ❌ No leading/trailing whitespace
- ✅ The full URL, exactly as Neon/Upstash provided

Click **Apply / Deploy Blueprint** at the bottom.

6.4 Watch the first build

Click into `qk-api` → **Logs** tab. Expected timeline (~5 min):

```
==__BODY__gt; Cloning from https://github.com/...
==__BODY__gt; Running build command __BODY__#39;npm ci --no-audit --include=dev __BODY__amp;__
added 349 packages in 11s
✔ Generated Prisma Client (v5.22.0)
__BODY__gt; tsc
==__BODY__gt; Build successful 🎉
==__BODY__gt; Deploying...
==__BODY__gt; Running __BODY__#39;cd backend __BODY__amp;__BODY__amp; npx prisma migrate deplo
Prisma schema loaded from prisma/schema.prisma
No pending migrations to apply.
Backend listening on http://localhost:4000
==__BODY__gt; Your service is live 🎉
```

When both services show **Live** (green dot), the backend is up. The API URL is at the top of the `qk-api` service page (e.g. `https://qk-api.onrender.com`).

6.5 Verify the API

From your laptop:

```
curl https://qk-api.onrender.com/health
# expected: {__BODY__quot;status__BODY__quot;:__BODY__quot;ok__BODY__quot;,__BODY__quot;timest
```

First request can take ~30s if the service was sleeping. Subsequent requests are fast.

Save the qk-api URL — Netlify needs it in step 7.

---

7. Step 5 — Deploy frontend on Netlify

7.1 Import the repo

1. Open https://app.netlify.com
2. **Add new site** → **Import an existing project**
3. Click **Deploy with GitHub** → authorise if prompted
4. Pick **quantumkaizen** from the repo list

7.2 Configure the build

Netlify auto-fills these from `netlify.toml`. Verify:

Field	Value
Branch to deploy	`__BODY__lt;DEPLOY_BRANCH__BODY__gt;` (same as Render)
Base directory	*(leave blank)*
Build command	`npm ci --no-audit __BODY__amp;__BODY__amp; npm run build --workspace=client`
Publish directory	`client/dist`

7.3 Add the API URL env var — BEFORE clicking Deploy

Vite reads `VITE_*` variables at **build time** and bakes them into the JS bundle. If you forget this step, the frontend tries to call its own domain for `/api/*` and breaks.

Scroll to **Environment variables** → **Add a variable** → **Add a single variable**:

Field	Value
Key	`VITE_API_BASE_URL`
Value	`https://qk-api.onrender.com/api` *(your Render URL from step 6.5, with `/api` appended)*
Scopes	Leave default (all deploy contexts)

Click **Create variable**.

7.4 Deploy

Click **Deploy [site-name]** at the bottom.

Netlify builds and deploys (~2–3 min). When status flips to **Published**, you get a URL like `https://chic-narwhal-abc123.netlify.app`.

Copy that URL — step 8 needs it.

---

8. Step 6 — Lock CORS to the Netlify origin

`CORS_ORIGIN=*` was OK for the initial deploy but is loose. Tighten it now.

1. Open https://dashboard.render.com → click **qk-api**
2. **Environment** tab → find `CORS_ORIGIN`
3. Click to edit → replace `*` with your exact Netlify URL
   - Example: `https://chic-narwhal-abc123.netlify.app`
   - **No trailing slash**
   - **Scheme matters** (`https://` exactly)

4. Click **Save Changes**

Render auto-redeploys `qk-api` (~2 min). When green, the frontend can talk to the backend and no other origin can.

---

9. Step 7 — Test end-to-end

Open your Netlify URL in the browser.

- [ ] Login page renders without console errors
- [ ] Log in with a seeded admin (or the user you created locally)
- [ ] Navigate to **Forms** tab → form list loads
- [ ] Click **Checklists** tab → list loads (empty if you haven__BODY__#39;t made any)
- [ ] Click **New checklist** → builder opens, can add a field, save, publish
- [ ] Network tab shows requests going to `https://qk-api.onrender.com/api/...` returning 200

If any of these fail, see **§11 Troubleshooting**.

---

10. Step 8 — Custom domain (optional)

10.1 Frontend on your domain (e.g. `quantumkaizen.com`)

1. Netlify dashboard → your site → **Site settings** → **Domain management** → **Add custom domain**
2. Enter your domain → Netlify shows DNS records to set:
   - `A` record for `@` (root) → Netlify load balancer IP
   - `CNAME` for `www` → `__BODY__lt;your-site__BODY__gt;.netlify.app`
3. At your registrar (Cloudflare, Namecheap, etc.), add those records
4. Wait 5–10 min for DNS propagation. Netlify auto-issues TLS via Let__BODY__#39;s Encrypt.

10.2 Backend on `api.yourdomain.com` (optional)

1. Render dashboard → **qk-api** → **Settings** → **Custom Domain** → **Add**
2. Enter `api.quantumkaizen.com` → Render shows a CNAME to set at your registrar
3. Add the CNAME → wait for verification
4. Once green:
   - Netlify → **Environment variables** → change `VITE_API_BASE_URL` to `https://api.quantumkaizen.com/api`
   - **Deploys** → **Trigger deploy** → **Deploy site** (Vite needs to rebuild with the new value)
   - Render → `qk-api` → **Environment** → change `CORS_ORIGIN` to your real frontend domain (e.g. `https://quantumkaizen.com`) → save

---

11. Day-2 operations

Redeploy

Push to `__BODY__lt;DEPLOY_BRANCH__BODY__gt;`. Both Render and Netlify auto-deploy on push.

Run a database migration

1. Edit `backend/prisma/schema.prisma` on your laptop
2. Generate the migration:

```
cd backend
DATABASE_URL=__BODY__#39;__BODY__lt;neon-pooled__BODY__gt;__BODY__#39; DIRECT_URL=__BODY__
  npx prisma migrate dev --name __BODY__lt;descriptive-name__BODY__gt;
```

3. Commit the new folder under `backend/prisma/migrations/`
4. Push → Render__BODY__#39;s `startCommand` runs `prisma migrate deploy` on every boot, so the next deploy applies it automatically

View logs

- **Backend**: Render → service → **Logs** tab (live tail)
- **Frontend build**: Netlify → site → **Deploys** → click any deploy → **Deploy log**
- **Frontend runtime**: browser DevTools console

Rollback

- **Backend**: Render → **Deploys** → pick an older green deploy → click → **Rollback to this deploy**
- **Frontend**: Netlify → **Deploys** → pick an older deploy → click **Publish deploy**

Wake a sleeping free service

First request after 15 min idle takes ~30 s. To eliminate cold starts, upgrade the Render service plan to **Starter** ($7/month) per service.

Rotate the JWT secret

This logs everyone out. Plan a maintenance window.

1. Generate a fresh secret: `openssl rand -base64 48`
2. Render → `qk-api` → **Environment** → edit `JWT_SECRET` → save
3. Repeat for `qk-worker`
4. Both services redeploy

12. Troubleshooting

Symptoms encountered during the initial deploy of this project

Build fails: `TS5107: Option __BODY__#39;moduleResolution=node10__BODY__#39; is deprecated`

The Render-side TypeScript treats the deprecation as an error. Local fix: remove the explicit `moduleResolution` field from `backend/tsconfig.json` — TypeScript will use the same default value (`node10`) without triggering the warning. Already applied in this repo (commit `bf09e9f`).

Build fails: `error TS7016: Could not find a declaration file for module __BODY__#39;cors__BODY__#39;`

Render sets `NODE_ENV=production` on the service, which makes `npm ci` default to `--omit=dev` , stripping `@types/*` and the Prisma CLI. The fix in `render.yaml` is `npm ci --no-audit --include=dev` . Already applied (commit `4f5f7a7`).

Runtime fails: `P1013: The provided database string is invalid`

`DATABASE_URL` or `DIRECT_URL` was pasted with quotes, with the `KEY=` prefix, or with the wrong scheme. Open Render → service → **Environment** → click the eye icon to reveal the value. It must start with `postgresql://` (no quotes, no `DATABASE_URL=` prefix) and end with `?sslmode=require` . Re-paste both URLs from Neon.

Other common issues

CORS error in the browser console

`CORS_ORIGIN` on `qk-api` doesn__BODY__#39;t exactly match the Netlify URL. Match scheme + host with no trailing slash:

```
Browser:  https://chic-narwhal-abc123.netlify.app
Render:   https://chic-narwhal-abc123.netlify.app   ✅
          https://chic-narwhal-abc123.netlify.app/  ❌ trailing slash
          chic-narwhal-abc123.netlify.app           ❌ missing scheme
```

API returns HTML instead of JSON in production

`VITE_API_BASE_URL` was missing at Netlify build time. Re-add the env var, then **Deploys → Trigger deploy → Deploy site** to rebuild Vite with the new value. The client logs an explicit `BACKEND_UNAVAILABLE` error when this happens — see `client/src/lib/api.ts` .

Worker isn__BODY__#39;t picking up jobs (BullMQ stuck)

Check `qk-worker` **Logs** for Redis connection errors. Most common cause: `REDIS_URL` was pasted with the scheme `redis://` instead of `rediss://` . Upstash requires TLS — must be `rediss://` (two s).

`prisma migrate deploy` fails on Render with __BODY__quot;shadow database__BODY__quot; error

You__BODY__#39;re accidentally running `migrate dev` , which needs a shadow DB. The `startCommand` should be `prisma migrate deploy` (no shadow DB needed). Verify in `render.yaml` .

App loads, but every API call hangs for 30s once a day

Render free tier cold start after 15 min idle. Upgrade `qk-api` to the $7/mo Starter plan to keep it always-on.

---

13. Reference — environment variables

qk-api (Render web service)

Variable	Source	Example
`NODE_ENV`	render.yaml literal	`production`
`PORT`	render.yaml literal	`4000`
`DATABASE_URL`	Neon pooled URL (step 4.2.A)	`postgresql://...@ep-x-pooler.neon.tech/neondb?...`
`DIRECT_URL`	Neon direct URL (step 4.2.B)	`postgresql://...@ep-x.neon.tech/neondb?...`
`REDIS_URL`	Upstash TCP URL (step 5.2)	`rediss://default:xxx@xxx.upstash.io:6379`
`JWT_SECRET`	`openssl rand -base64 48` (step 6.1)	64-char base64 string
`JWT_EXPIRES_IN`	render.yaml literal	`7d`
`CORS_ORIGIN`	Netlify URL (step 8)	`https://your-site.netlify.app`

qk-worker (Render background worker)

Same as `qk-api` except: no `PORT`, no `CORS_ORIGIN`, no `JWT_EXPIRES_IN`.

Netlify (frontend build)

Variable	Source	Example
`VITE_API_BASE_URL`	Render API URL + `/api`	`https://qk-api.onrender.com/api`

---

14. Files in this repo that control the deploy

File	Purpose
`render.yaml`	Render Blueprint — defines `qk-api` and `qk-worker` services
`netlify.toml`	Netlify build config — install + build command, SPA redirects
`vercel.json`	Alternate frontend config for Vercel (unused if deploying to Netlify)
`backend/prisma/`	Migrations + schema applied on every Render boot
`backend/src/lib/env.ts`	Validates all backend env vars at startup (Zod schema)

When updating the deployment, edit these files locally, commit, and push — both Render and Netlify pick up the changes on next deploy.

End of guide.